

CYBERSECURITY & CRYPTOGRAPHY

Weekly Important Study Notes • Simple Theory + Practical Understanding

Topics covered: Cryptography fundamentals, encryption, AES, symmetric encryption, asymmetric encryption, hashing, digital signatures, HTTPS/TLS, certificates, CyberChef, Python, and Wireshark HTTPS traffic analysis.

Topic	Remember This
Encryption	Converts plaintext into ciphertext using a key; designed to protect confidentiality.
AES	A symmetric block cipher; the same secret key is used for encryption and decryption.
Asymmetric crypto	Uses a public key and a private key; useful for secure key exchange and digital signatures.
Hashing	One-way transformation used for integrity checks and password storage; not encryption.
Digital signature	Uses a private key to sign and a public key to verify authenticity and integrity.
HTTPS/TLS	Protects data in transit between a client and server.
Wireshark	Captures and analyzes network packets; HTTPS payload is normally encrypted.
CyberChef	Web-based tool for learning and performing common encoding, hashing, and crypto operations.
Python	Can automate hashing, encryption experiments, file processing, and security tasks.

1. Cryptography Fundamentals

What is cryptography?

Cryptography is the practice of protecting information by transforming it so that unauthorized people cannot understand or modify it.

Main security goals

Confidentiality: only authorized people can read data.

Integrity: data should not be changed without detection.

Authentication: verify who a user or system is.

Non-repudiation: provide evidence that a digital action/signature came from a particular key holder.

Basic terms

Plaintext = original readable data. **Ciphertext** = transformed/encrypted data. **Key** = secret or cryptographic value controlling the operation.

2. Encryption Concepts

Encryption workflow

Plaintext → Encryption algorithm + key → Ciphertext → Decryption algorithm + key → Plaintext.

Encryption vs hashing

Encryption is designed to be reversible when the correct key is available. Hashing is designed as a one-way transformation and is mainly used for integrity checks and password verification.

Key protection

A strong algorithm is not enough if the key is exposed. Keys should be protected, access-controlled, rotated when appropriate, and never hard-coded into public source code.

3. Symmetric Encryption

Shared-key encryption

Symmetric encryption uses the same secret key for encryption and decryption. Both sides therefore need access to the secret key.

AES

AES (Advanced Encryption Standard) is a widely used symmetric block cipher. Common AES key sizes are 128, 192, and 256 bits. AES encrypts fixed-size blocks and is normally used with a secure mode of operation.

Simple AES idea

Sender: plaintext + secret key → ciphertext. Receiver: ciphertext + same secret key → plaintext.

Practical uses

File encryption, disk encryption, secure application data, VPNs, and protected network sessions.

Important security point

Do not reuse keys carelessly, do not invent your own encryption algorithm, and use established libraries and secure modes such as authenticated encryption where appropriate.

4. AES Encryption Workflow / Boxes

Typical learning diagram

Plaintext → Key → AES Encryption → Ciphertext. On the receiving side: Ciphertext → Same AES Key → AES Decryption → Plaintext.

What each box means

Plaintext: original message/file. **Key:** secret value. **AES Encryption:** mathematical transformation using the key.

Ciphertext: protected unreadable output. **AES Decryption:** reverses the encryption when the correct key and parameters are supplied.

IV / nonce

Many AES modes use an IV or nonce. It is a parameter used with the encryption operation and is not generally the same thing as the secret key. Correct uniqueness and handling depend on the mode.

5. Asymmetric Encryption

Public and private keys

Asymmetric cryptography uses a key pair. The **public key** can be shared. The **private key** must be kept secret.

Key exchange

Asymmetric cryptography can help establish or protect shared secrets over an untrusted network. Modern protocols commonly use public-key techniques during connection setup and then use symmetric encryption for bulk data.

Digital trust

A public key by itself does not automatically prove who owns it. Certificates and trusted certificate authorities help bind a public key to a domain or identity.

Applications

TLS, secure key exchange, digital signatures, certificates, secure email, and identity verification.

6. Hashing

Hash function

A hash function takes input of arbitrary size and produces a fixed-length digest. A small change in input should normally produce a substantially different digest.

Password hashing

Passwords should not normally be stored as plaintext. Password systems use dedicated password-hashing algorithms with salts and appropriate work factors. Fast general-purpose hashes such as SHA-256 are not by themselves ideal password-storage schemes.

Data integrity

A hash can be calculated for a file or message and later recalculated. Matching digests provide evidence that the content is unchanged, assuming the comparison value itself is trustworthy.

Hash verification

Input → hash function → digest. Later: same input → hash function → digest. Compare the expected and calculated digests.

Collision awareness

A collision occurs when two different inputs produce the same hash output. Cryptographic hash functions are designed to make useful collisions computationally difficult. Older algorithms such as MD5 and SHA-1 have known collision weaknesses and should not be selected for new security designs.

7. Digital Signatures

Purpose

Digital signatures help provide authenticity, integrity, and non-repudiation properties for signed digital data.

Simple workflow

Data → hash → sign the digest with the private key → digital signature. The verifier uses the public key to verify the signature and checks the data's integrity.

Signature vs encryption

A digital signature is mainly about proving authenticity/integrity. Encryption is mainly about confidentiality. They solve different security problems and can be used together.

Trust models

Certificate-based trust commonly relies on trusted certificate authorities. Other systems can use different trust models, such as a web of trust or manually trusted keys.

8. Secure Communications: HTTPS, TLS and SSL

HTTPS

HTTPS is HTTP carried through a secure TLS connection. It helps protect confidentiality and integrity of application data in transit and authenticates the server through certificates.

TLS

TLS is the modern protocol family used to secure many network connections. A TLS connection negotiates security parameters, authenticates the server where applicable, establishes keying material, and then protects application traffic.

SSL history

SSL is the older predecessor to TLS. Modern secure deployments should use current supported TLS versions rather than obsolete SSL versions.

Certificate concepts

A certificate can contain a subject/domain identity, public key, validity information, issuer information, and a digital signature from the issuer. Browsers validate certificate chains and other constraints.

9. TLS Handshake — Simple Workflow

Step-by-step

1) Client starts a secure connection. 2) Server responds with supported security information and its certificate. 3) Client validates the certificate and establishes keying material using the negotiated protocol. 4) Both sides derive session keys. 5) Application data is then encrypted and integrity-protected.

Certificate validation

Important checks include certificate validity, hostname/domain matching, trust chain, and applicable usage/constraints. A certificate being present is not by itself proof that every aspect of a connection is secure.

Secure browsing

When you visit an HTTPS website, the browser and server establish a TLS-protected channel before normal application data is exchanged.

10. Wireshark — Analyze HTTPS Traffic

What Wireshark does

Wireshark captures and displays network packets so you can study protocols, addresses, ports, packet timing, and connection behavior.

Basic practical workflow

1) Open Wireshark. 2) Select the active network interface. 3) Start capture. 4) Visit an HTTPS website for a controlled test. 5) Stop capture. 6) Filter traffic using terms such as **tls**, **tcp**, or a known server IP. 7) Inspect the TLS handshake and packet details.

What to look for

Look for TCP connection setup, TLS ClientHello, ServerHello, certificate-related handshake information, encrypted application data, packet lengths, timing, source/destination addresses, and ports.

Important limitation

For a normal modern HTTPS connection, Wireshark can show metadata and handshake details but cannot simply display the protected webpage content because the application payload is encrypted.

Learning goal

Understand the difference between visible network metadata and encrypted application content, and identify where TLS begins protecting the communication.

11. CyberChef Practical Learning

What CyberChef is

CyberChef is a browser-based tool used for learning and performing many data transformations, encoding/decoding operations, hashing operations, and cryptographic experiments.

Hash workflow

Input text/file → select a hash operation such as SHA-256 → run the recipe → compare the resulting digest with another trusted digest.

Encryption learning

You can use CyberChef to observe how plaintext becomes ciphertext and how the correct key/settings are required to recover the original data. Always distinguish encoding from encryption.

Good practice

Use CyberChef mainly as a learning and testing aid. For real applications, use well-maintained cryptographic libraries and secure protocol implementations.

12. Python for Cryptography & Security

Why Python?

Python can automate repetitive security tasks, calculate hashes, process files, analyze logs, and use established security libraries.

Simple hashing example

```
import hashlib
text = b"hello"
digest = hashlib.sha256(text).hexdigest()
print(digest)
```

What the code does

The data is converted to bytes, SHA-256 calculates a digest, and `hexdigest()` displays the digest as readable hexadecimal text.

Security rule

Do not create custom cryptographic algorithms. Use established libraries, secure defaults, safe key handling, and current protocol recommendations.

13. Important Differences for Revision

Encryption vs Hashing

Encryption: reversible with the required key; protects confidentiality. Hashing: designed as one-way; supports integrity and password verification.

Symmetric vs Asymmetric

Symmetric: one shared secret key; efficient for bulk data. Asymmetric: public/private key pair; useful for identity, signatures, and establishing shared secrets.

Hash vs Digital Signature

Hash gives a digest. A digital signature adds a private-key-based authenticity mechanism to signed data/digests.

HTTP vs HTTPS

HTTP does not provide TLS protection by itself. HTTPS uses HTTP over a TLS-protected connection.

Encoding vs Encryption

Encoding changes representation for compatibility or transport and is not intended to provide secrecy. Encryption is specifically designed to protect confidentiality.

14. Exam / Interview Important Points

Remember these one-line answers

- AES = symmetric encryption algorithm.
- SHA-256 = cryptographic hash function producing a 256-bit digest.
- Public key = shareable key in asymmetric cryptography.
- Private key = secret key that must be protected.
- Hash = one-way digest for a given input.
- Digital signature = private-key signing + public-key verification.
- HTTPS = HTTP protected using TLS.
- TLS certificate = helps authenticate a server/domain and provides its public key.
- Wireshark = packet capture and protocol analysis tool.
- CyberChef = interactive data transformation and security learning tool.

Common mistakes to avoid

Do not say hashing is encryption. Do not treat Base64 as encryption. Do not share private keys. Do not assume HTTPS makes every website trustworthy. Do not use obsolete SSL versions for modern secure designs.

15. Quick Practical Checklist

Practice 1 — Hash

Create a text string, calculate SHA-256 in CyberChef, then calculate SHA-256 in Python and compare the digest.

Practice 2 — Encryption

Use a controlled test message in CyberChef to understand an encryption/decryption workflow and identify the role of the key.

Practice 3 — Wireshark

Capture your own HTTPS browsing session, filter for TLS, and identify ClientHello, ServerHello, certificate-related information, and encrypted application data.

Practice 4 — Explain

Practice explaining AES, hashing, digital signatures, certificates, and HTTPS in 2–3 sentences each.

FINAL REVISION MAP

Cryptography → protects information using mathematical techniques.

↓

Encryption → confidentiality → AES / asymmetric cryptography

↓

Hashing → integrity / password verification → SHA-256 and password-hashing concepts

↓

Digital Signatures → authenticity + integrity + non-repudiation

↓

TLS / HTTPS → secure communication over networks

↓

Wireshark + CyberChef + Python → practical analysis, experimentation, and automation

Study tip: First understand the purpose of each technology, then memorize the workflow. For practical work, always test with your own data or an authorized lab environment.